AdaptArena v2: does training where the policy fails most survive unseen shifts better?

Technical report · results tier **full** · analysis plan SHA-256 `936d3e3909573fa9…` · x86_64 · torch 2.14.1+cu130, numpy 2.5.4

1. Question

Does sampling training scenarios by *planner regret* (reference-planner survival minus policy survival) produce agents that survive unseen shifts better than uniform randomization, at equal transition budget? This is a hypothesis, and the report is written so that either answer is reportable. Arms: A specialist (one fixed layout), B uniform randomization (plain domain randomization, [Tobin et al., Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World, IROS 2017]), C replay prioritised by positive value loss, D replay prioritised by planner regret; C and D share one buffer implementation, so the score is the only difference. Two comparisons were declared in advance (D vs B, D vs C); everything else is exploratory.

Positioning. Minigrid [Chevalier-Boisvert et al., Minigrid & Miniworld, NeurIPS Datasets and Benchmarks 2023] and Procgen [Cobbe et al., Leveraging Procedural Generation to Benchmark Reinforcement Learning, ICML 2020] are established generalisation benchmarks and Craftax [Matthews et al., Craftax: A Lightning-Fast Benchmark for Open-Ended Reinforcement Learning, ICML 2024] is a fast survival-style benchmark. What this project adds is narrower: a full-information reference planner that yields solvability certificates and exact regret on every scenario, bit-exact integer simulator state, controlled interventions with stress curves, and a locked statistical protocol. Replay-guided environment design is not new: Prioritized Level Replay uses a value-loss score as a regret proxy [Jiang, Grefenstette, Rocktäschel, Prioritized Level Replay, ICML 2021; Jiang et al., Replay-Guided Adversarial Environment Design, NeurIPS 2021], and PAIRED uses regret directly [Dennis et al., Emergent Complexity and Zero-shot Transfer via Unsupervised Environment Design, NeurIPS 2020]. Here regret is cheap because the world is small, which is a limitation of the setting rather than evidence for a general method. This is not a new RL algorithm and no general claim about regret-guided training is made.

2. Environment

`AdaptArena-Survival-v1`: a 15×15 grid, six actions, reward 1 per living step, 500-step horizon. Energy and hydration are integers in hundredths (0 to 10000), drains are integers and cost multipliers are integer percents, so NumPy, a future JAX port and every machine produce identical SHA-256 state hashes. The observation is a 9×9 agent-centred local view with 8 channels plus an 11-value stats vector (energy, hydration, cold, previous action one-hot, previous action valid, energy drained last step). Invalid moves pay the movement surcharge, invalid interactions only the base drain; simultaneous death causes are all stored and the primary reason follows hazard, dehydration, starvation. A full-information search **planner** sees the map, respawn and weather schedules and returns a plan that survives 500 steps; each plan is replayed in the environment to verify it, and on tiny maps the planner is cross-checked against exhaustive search. A scenario enters any suite only if the planner certifies it. The planner is a strong reference, never called optimal. Throughput on Intel(R) Xeon(R) Processor @ 2.80GHz (4 cores): 18,499 steps/s for the scalar reference implementation and 32,158 (8 envs) to 186,942 (256 envs) steps/s for the vectorised batch environment, including observation construction; every experiment budget below was sized against these numbers.

Difficulty calibration (validation seeds only, before any learned policy ran; constants frozen in the analysis plan):

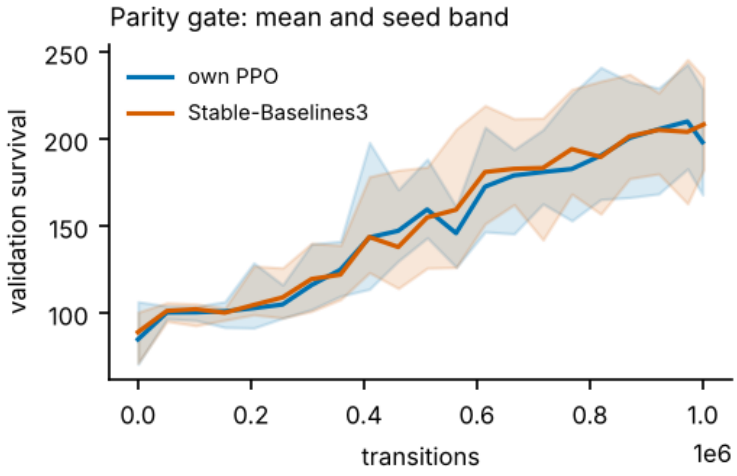
Condition (validation)	random	reactive script	memory script	planner
familiar	105	395	500	500
unseen_layout	87	361	500	500
scarce_food	89	196	500	500
scarce_water	88	183	467	500
higher_move_cost	71	352	500	500
cold_snap	88	238	471	500
combined_shift	71	107	425	500

3. Methods

Learner. PPO written from scratch in one file (Huang et al., CleanRL: High-quality Single-file Implementations of Deep RL Algorithms, JMLR 2022 style; Schulman et al., Proximal Policy Optimization Algorithms, 2017; Schulman et al., High-Dimensional Continuous Control Using Generalized Advantage Estimation, ICLR 2016): vectorised rollouts, GAE with bootstrap on truncation and never on death, clipped surrogate, entropy bonus, advantage normalisation, linear learning-rate decay, gradient clipping, and a recurrent variant on 64-step sequence chunks with hidden-state resets at episode boundaries (the stretch SSM is a small Mamba-style selective state-space block [Gu and Dao, Mamba: Linear-Time Sequence Modeling with Selective State Spaces, 2023], trained with the same loop; it was not run in this tier). It is validated by unit tests (hand-computed GAE with one death and one truncation, ratio 1 and clip fraction 0 on the first epoch, overfitting one batch, a tiny solvable environment, same-seed reproducibility, LSTM and SSM sequence pass equal to the step-by-step pass) and by the parity gate against Stable-Baselines3 [Raffin et al., Stable-Baselines3, JMLR 2021]. Tuned (learning rate, entropy coefficient) on the feedforward sweep: 0.001, 0.003. The best cell sits on the

edge of the grid and the 300k-transition sweep favours fast starters, a limitation of the protocol. The LSTM sweep was uninformative (every cell within noise of an untrained policy: validation survival 100 to 107), so by a dated amendment recorded before any memory-track run the LSTM and SSM use the feedforward pair; this is a recorded difference in tuning effort, and the memory-track comparison is not called controlled with respect to tuning.

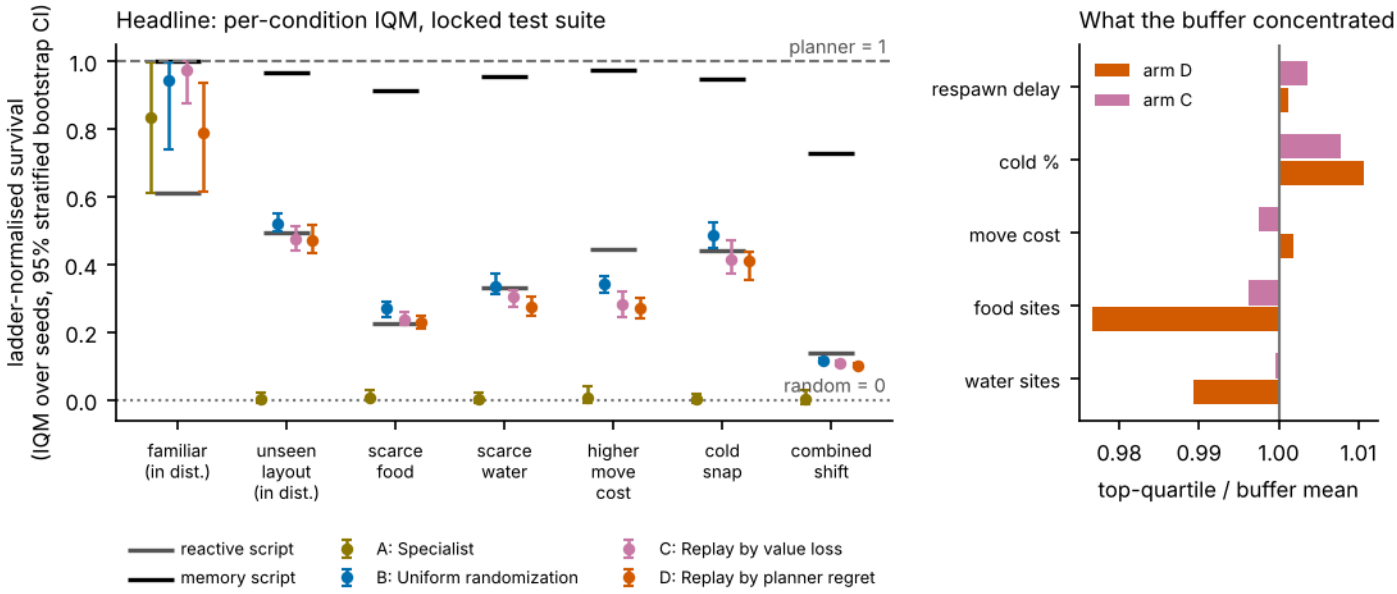
Parity gate with SB3. Both implementations trained with the same hyper-parameters, environment, reward scaling and transition budget on 10 seeds each. Final IQM survival: own 216.5 [203.1, 229.0], SB3 215.6 [201.5, 227.2]; each lies inside the other's interval; mean curves overlap the other's seed band at 100% of evaluation points (threshold 80%). **Gate passed.**



Learning curves (validation survival), mean and min-max seed band.

Protocol. 10 seeds per arm at 2,000,000 transitions each (counted across all environments). Evaluation uses the locked test suite: 7 conditions × 50 planner-certified scenarios, deterministic actions as the primary mode, identical scenarios for every arm. Statistics follow Agarwal et al., Deep Reinforcement Learning at the Edge of the Statistical Precipice, NeurIPS 2021: IQM with stratified bootstrap confidence intervals, optimality gap against the planner, probability of improvement, performance profiles, with the training seed within a condition as the unit of analysis. The analysis plan (hypotheses, arms, seed counts, metrics, thresholds, parity criteria, memory predictions) was hashed (`936d3e3909573fa9…`) and committed to git before the test suite was generated; the harness refuses to open the suite otherwise and logs every access. Checkpoints are chosen by highest mean validation survival, earlier wins ties.

4. Results



Headline figure. Left: per-condition IQM with 95% CIs for the four arms against the random floor, the scripted baselines and the planner ceiling. Right: the companion panel showing what the replay buffers concentrated on (top-priority quartile relative to the buffer mean).

Pre-declared outcome. Uniform randomization wins here; see the diagnosis of why regret misled. (rule fired: `D_loses_to_B`). No condition was saturated by every arm, so no condition is excluded for saturation.

Arm	Aggregate IQM [95% CI]	Optimality gap	Training cost
A: Specialist	0.01 [0.00, 0.02]	0.88	20 min/run
B: Uniform randomization	0.39 [0.37, 0.40]	0.58	18 min/run
C: Replay by value loss	0.34 [0.32, 0.35]	0.60	17 min/run
D: Replay by planner regret	0.32 [0.30, 0.33]	0.64	18 min/run, 4894 planner calls (22 s)

Comparison	IQM difference [CI]	P(improvement) [CI]	Claim status
D vs B (pre-declared)	-0.07 [-0.09, -0.05]	0.19 [0.12, 0.27]	not supported
D vs C (pre-declared)	-0.02 [-0.04, -0.00]	0.38 [0.28, 0.48]	not supported

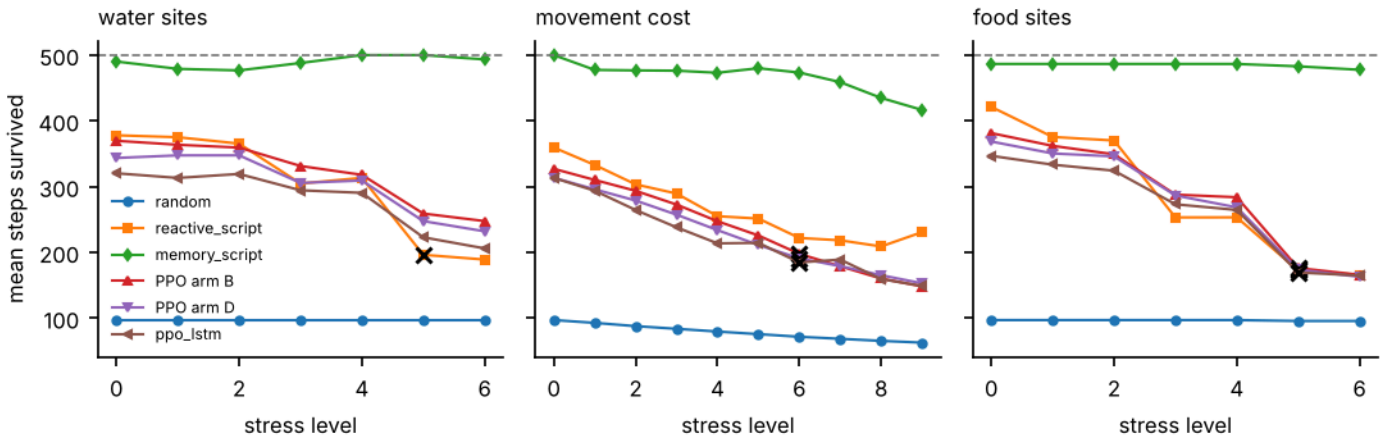
Scores are ladder-normalised survival (random = 0, planner = 1), averaged over episodes inside each seed first; 10 training seeds per arm, 50 certified scenarios per condition, stratified bootstrap with 10,000 resamples. Every comparison that was run, including exploratory ones, is in the appendix table `comparisons_appendix.csv`.

Secondary robustness check, sampled actions (kept in its own file and never mixed into the aggregate above): arm A IQM 0.017; arm B IQM 0.492; arm C IQM 0.458; arm D IQM 0.431. The ordering of the arms is unchanged.

Under the combined shift, the D−B difference is -0.02 [-0.02, -0.00] (exploratory). Training cost for arm D includes planner calls; every arm used the same transition budget.

Diagnosis of why regret did not help (exploratory; descriptive numbers from the headline runs and fresh rollouts, not part of the pre-declared comparisons). (1) In this training distribution the planner certified 100% of sampled scenarios, so planner survival is the 500-step cap everywhere and regret reduces to the policy's shortfall; there were no unsolvable scenarios for the planner to steer the curriculum away from, which is the one thing a planner adds over a value-loss score. (2) At equal budget, 48% of arm D's episodes (4604 replays vs 4894 fresh scenarios on average) and 48% of arm C's replayed stored scenarios, so each replay arm trained on roughly half as many distinct scenarios as arm B. (3) What counts as hard is not a property the scenario parameters reveal: a gradient-boosted model of survival on the six scenario parameters has cross-validated R^2 of 0.02, and the top-priority quartile of the buffer is within 3% of the buffer mean on every parameter (companion panel). Hardness lives in layout geometry and timing. (4) The score is noisy: the rank correlation of one scenario's survival across two independent sampled rollouts is 0.60 (range 0.54 to 0.64), so roughly a third or more of the ranking is action-sampling noise. Arm D's training-time episode survival (291) is below arm B's (372), consistent with replay concentrating on scenarios the policy fails. A reading consistent with these numbers is that prioritising shortfall in a fully solvable, high-variance distribution spends half the budget re-visiting a few idiosyncratic hard layouts at the cost of diversity; this was not tested with a controlled intervention (for example a lower replay probability) and should be read as a hypothesis.

5. What it learned



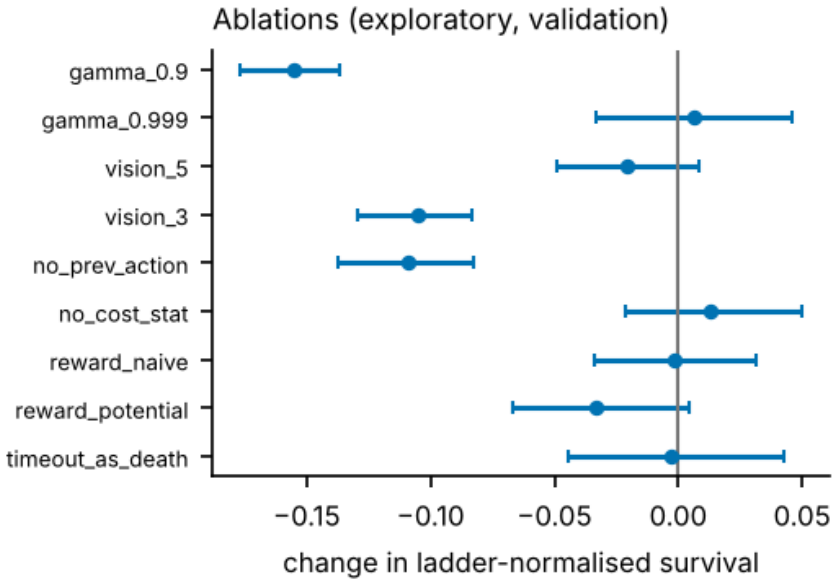
Stress curves for every rung (marker × = breaking point: first level below half of familiar-condition survival). Saturated levels and unsolvable levels are drawn and flagged on the site and in `stress.json`.

Breaking points: memory_script (water): none within 10 levels; ppo_ff_armB (water): none within 10 levels; ppo_ff_armD (water): none within 10 levels; ppo_lstm (water): none within 10 levels; memory_script (cost): none within 10 levels; ppo_ff_armB (cost): level 6; ppo_ff_armD (cost): level 6; ppo_lstm (cost): level 6.

Probes (each paired with what it does not show). Hydration threshold: constructed states with the stat overwritten, so behaviour off-distribution. Distilled depth-3 tree agrees with the policy on 37% of held-out states (tree survival 80 vs policy 382); agreement is approximation, not the policy's logic. Occlusion heatmaps show input sensitivity only; the training time machine shows when behaviours appear, not why. The point-of-no-return

probe forks recorded states every 25 steps and asks the planner for a certificate; it is valid for the planner's search power, not a proof that no plan exists.

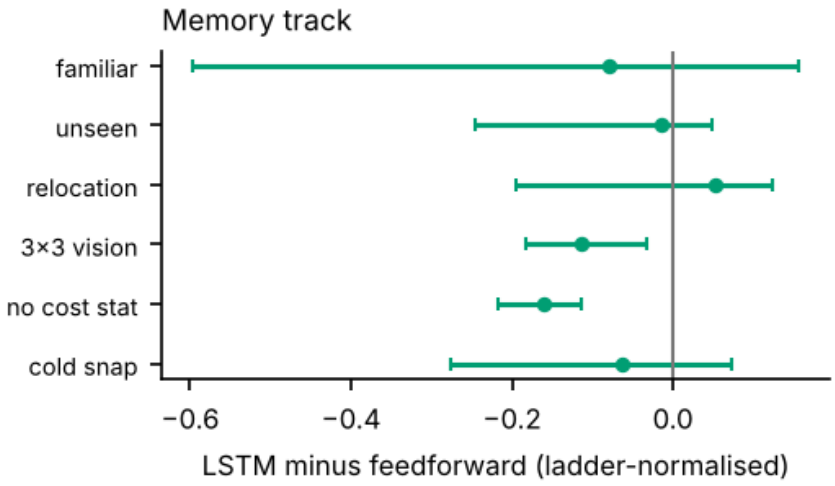
Red team. reactive_script: worst certified-solvable scenario scored -0.01 (survival 73); memory_script: worst certified-solvable scenario scored 0.05 (survival 91); ppo_ff_armB_seed0: worst certified-solvable scenario scored -0.08 (survival 71); ppo_ff_armD_seed0: worst certified-solvable scenario scored -0.11 (survival 56) after 400+ candidates each; the five most instructive are in the failure museum with templated narration (no language model).



Single-change ablations, exploratory, validation suite only.

Ablation effects (ladder-normalised survival vs default, 5 seeds): gamma_0.9: -0.155 [-0.177, -0.137] (hurts); gamma_0.999: 0.007 [-0.033, 0.046] (no measurable effect (interval includes zero)); vision_5: -0.021 [-0.049, 0.008] (no measurable effect (interval includes zero)); vision_3: -0.105 [-0.130, -0.083] (hurts); no_prev_action: -0.109 [-0.138, -0.083] (hurts); no_cost_stat: 0.013 [-0.021, 0.050] (no measurable effect (interval includes zero)); reward_naive: -0.002 [-0.034, 0.032] (no measurable effect (interval includes zero)); reward_potential: -0.033 [-0.067, 0.004] (no measurable effect (interval includes zero)); timeout_as_death: -0.003 [-0.044, 0.043] (no measurable effect (interval includes zero)).

Naive shaping, exploit check: the fraction of valid interactions taken with a stat at 90% or more was 26% under the naive per-interaction bonus, 27% under potential-based shaping and 29% under the sparse reward: no sign of the exploit by this metric.



LSTM minus feedforward, per shift, with 95% CIs.

Shift	Locked prediction (recurrent vs feedforward)	Scored
familiar and unseen layouts	recurrent ahead	**miss**
resource relocation	recurrent loses most of its lead	**miss**
vision reduced to 3x3	recurrent clearly ahead	**miss**

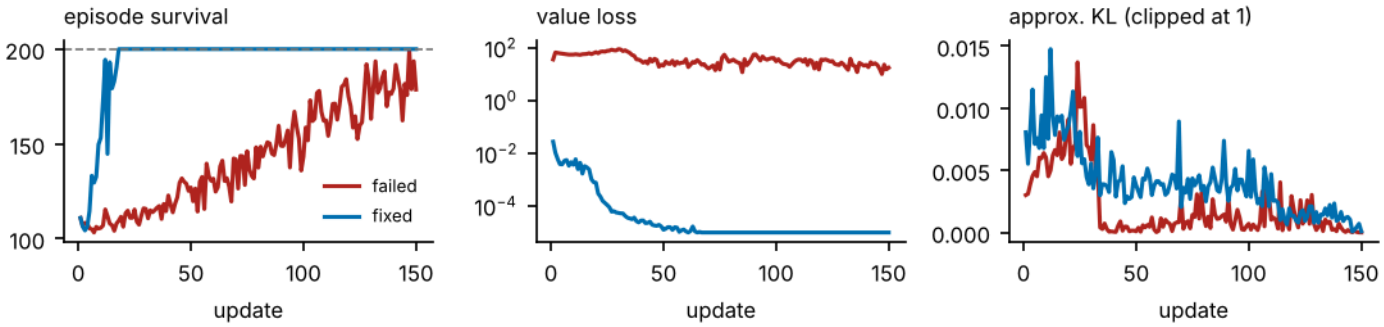
movement-cost increase, energy-cost stat removed	recurrent holds survival better	**miss**
cold snap	no separation	**hit**

1 of 5 locked predictions were hits; the rest are scored as written, without revision. The recurrent policy did not beat the feedforward policy where memory was predicted to help. Best-checkpoint validation survival was 232 for the LSTM (5 seeds) vs 263 for the feedforward policy (10 seeds). Caveats that bear on how to read this: LSTM and SSM share the feedforward-selected hyper-parameters (see the amendment above), recurrent training uses 64-step truncated chunks, the LSTM has 5 seeds against 10, and a recurrent policy may need more than the 2M-transition budget the feedforward policy was sized for. The result says the memory hypothesis was not supported at this budget, not that memory cannot help.

Recurrent-state probe (held-out episodes, shuffled-label control): steps-since-water R^2 -0.28 from the hidden state vs -0.03 shuffled and 0.00 from the current observation alone; direction accuracy 0.43 vs 0.29 shuffled (majority class 0.31). This shows whether memory holds task information, and nothing more.

Int8 post-training quantization (stretch, exploratory, measured on Intel(R) Xeon(R) Processor @ 2.80GHz only): ppo_ff: retention IQM 0.99 [0.97, 1.01], 1623 → 415 kB, latency 0.34 → 0.30 ms (batch 8, 1 thread); ppo_lstm: retention IQM 1.01 [0.95, 1.07], 1702 → 436 kB, latency 0.47 → 0.44 ms (batch 8, 1 thread).

The debugging story



A real failed run and its fix, regenerated from logged diagnostics on a tiny solvable scenario whose optimal return is known (planner survival 200).

The first PPO runs sat near random-level survival on a scenario the planner solves. The diagnostics pointed at the optimiser, not the environment: the value loss was 57 on average over the first 20 updates (vs 0.005 after the fix) while entropy barely moved. With a reward of 1 per step and γ = 0.99, returns are around 100, so the value-loss gradient dominated the single joint gradient-norm clip (0.5) and left the policy gradient almost nothing. Scaling the learner's reward by 0.01 (the environment and every reported survival number are untouched) solved the scenario in 18 updates, whereas the unscaled run reached 179 of 200 after 150 updates. The very first failing run (which also showed approximate-KL spikes above 6) was observed during development and not retained; the figure is the regenerated version of the same comparison, which reproduces the failure mode as slow learning rather than the spikes. Kept in `results/<tier>/debug/`.

6. Limitations

- One small custom environment. Any statement about regret-guided training holds only under this benchmark, with these limits; nothing is claimed for domain randomization in general.
- Regret is cheap here because a planner exists. In most environments it is unobservable.
- The planner is a depth-first search over resource-visit macros with a node cap and hazards treated as walls; a scenario without a certificate may still be solvable. It is a reference, not an optimum.
- Probes show behaviour and sensitivity, not reasoning; heuristic labels (blind or blunder) are suggestions.
- Compute: 10 seeds × 2,000,000 transitions per arm; ablations 5 seeds × 600,000; memory track 5 seeds (LSTM, 2,000,000) and 5 seeds for the 3×3-vision and no-cost-stat variants (1,000,000); tuning sweep at 300,000 transitions with 3 seeds per cell. These sizes were fixed before results existed. Training reproducibility is same machine and same software; evaluating a fixed checkpoint is bit-exact.
- Hardware and speed statements apply to the named machine only. No claim is made about SSM or quantized hardware speed or energy beyond what was measured there.
- The Play page uses Pyodide with the same Python core; it was tested in headless Chromium, not across browsers or phones.

7. Reproducibility

`make reproduce TIER=smoke|dev|full` runs every stage behind its gate (smoke: minutes on a laptop, 1 seed, in CI on every commit; dev: 3 seeds, 20 scenarios per condition; full: the published benchmark). Config hashes and checksums are in `manifest.json`; each stage writes a resumable results folder with manifests, episode CSVs, aggregate CSVs, training metrics, replays and a lockfile. Claims in the ledger link to their episode data. Policy weights are omitted from the repository except released checkpoints (model cards in `releases/`).

References

1. Chevalier-Boisvert et al., Minigrid & Miniworld, NeurIPS Datasets and Benchmarks 2023
2. Cobbe et al., Leveraging Procedural Generation to Benchmark Reinforcement Learning, ICML 2020
3. Matthews et al., Craftax: A Lightning-Fast Benchmark for Open-Ended Reinforcement Learning, ICML 2024
4. Jiang, Grefenstette, Rocktäschel, Prioritized Level Replay, ICML 2021
5. Jiang et al., Replay-Guided Adversarial Environment Design, NeurIPS 2021
6. Dennis et al., Emergent Complexity and Zero-shot Transfer via Unsupervised Environment Design, NeurIPS 2020
7. Agarwal et al., Deep Reinforcement Learning at the Edge of the Statistical Precipice, NeurIPS 2021
8. Huang et al., CleanRL: High-quality Single-file Implementations of Deep RL Algorithms, JMLR 2022
9. Raffin et al., Stable-Baselines3, JMLR 2021
10. Schulman et al., Proximal Policy Optimization Algorithms, 2017
11. Schulman et al., High-Dimensional Continuous Control Using Generalized Advantage Estimation, ICLR 2016
12. Gu and Dao, Mamba: Linear-Time Sequence Modeling with Selective State Spaces, 2023
13. Tobin et al., Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World, IROS 2017